

Jesus Librado Chavez Huerta

Data Engineer | Pipelines, Medallion Architecture & Data Quality

***Ubicación:** Agua Prieta Sonora México*

***Correo:** jesus_chh@hotmail.com*

Sobre Mi: Ingeniero de datos enfocado en el diseño de arquitecturas Medallion y procesamiento de datos bajo tipos de datos estructurados y semi- estructurados.

Stack Tecnológico:

Python, PySpark, SQL, T-SQL, SQL Spark, KQL, Cloud Azure, Delta Lake, Lake House, WareHouse, EventHouse, Semantic Model, Pipelines.

Herramientas: Microsoft Fabric, ADF, Dataflow Gen2, Notebooks, Semantic Model, Arquitectura Medallion, Almacenamientos.

Fabric

smart_device_analythcs_ws

Área de trabajo para el curso de Fabric en Proyecto 1

Development

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

Filtrar por palabra d...

Filtro

Bronze Data

Agregar tarea

Agregar un conector

Aplicar flujo de tareas prediseñado

Process Get Data

Obtener datos

3 elementos

+ Nuevo elemento

Bronze Data

Almacenar datos

2 elementos

+ Nuevo elemento

Ingestion Process

Preparación de datos

9 elementos

+ Nuevo elemento

Silver Data

Almacenar datos

3 elementos

+ Nuevo elemento

Transform Process

Preparación de datos

9 elementos

+ Nuevo elemento

Gold Data

Almacenar datos

2 elementos

+ Nuevo elemento

Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
Copy Job		Carpeta	—	—	—	—	—	—	
DataFlow_Gen2		Carpeta	—	—	—	—	—	—	
Notebook		Carpeta	—	—	—	—	—	—	
Pipeline		Carpeta	—	—	—	—	—	—	

Estructura centralizada de almacenamiento: separación por capas (Bronze, Silver, Gold) para asegurar trazabilidad y calidad de la información.

Fabric

smart_device_analythcs_ws
Área de trabajo para el curso de Fabric en Proyecto 1

Ver canalización de implementación Crear aplicación Administrar acceso Configuración del área de trabajo

+ Nuevo elemento Nueva carpeta Importar Migrar

Filtrar por palabra cl... Filtro

Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
Pipeline		Carpeta	—	—	—	—	—	—	
lh_bronze		Lakehouse	Bronze D...	JESUS LIBRA...	—	—	—	—	
lh_bronze		Punto de con...	Bronze D...	JESUS LIBRA...	—	—	—	—	
lh_gold		Lakehouse	Gold Data	JESUS LIBRA...	—	—	—	—	
lh_gold		Punto de con...	Gold Data	JESUS LIBRA...	—	—	—	—	
lh_silver		Lakehouse	Silver Data	JESUS LIBRA...	—	—	—	—	
lh_silver		Punto de con...	Silver Data	JESUS LIBRA...	—	—	—	—	
sm_smarth_device_lh		Modelo semá...	—	smart_device...	17/8/2026, 11:2...	N/D	—	—	
StagingLakehouseForDataflows_202607310412:		Lakehouse	—	JESUS LIBRA...	—	—	—	—	
StagingLakehouseForDataflows_202607		Punto de con...	—	JESUS LIBRA...	—	—	—	—	
StagingWarehouseForDataflows_202607310412		Warehouse	—	JESUS LIBRA...	—	—	—	—	
wh_smart_device		Warehouse	—	JESUS LIBRA...	—	—	—	—	

Estructura de organización en Workspace “*smart device analytics*” donde se distribuye la arquitectura medallion para sus diversas funciones asegurando la correcta aplicación por capa del dato.

Fabric lh_bronze X

Inicio Vistas de lago materializadas

Obtener datos Nueva consulta de SQL de Spark Nuevo modelo semántico Agregar al agente de datos Administrar la seguridad de OneLake Actualizar todas las variables

Explorador

Agregar almacenes de lago de datos

lh_bronze

- Tables
 - dbo
- Files
 - 2025-07-07
 - 2025-07-14
 - 2025-07-21

camera

Vista de tabla

Mostrando 1000 filas

	123	camera_id	ABC camera_resolution	ABC effective_pixel	ABC image_format	ABC videos_recording	ABC flash	ingestion_date	ABC enviroment
1	10001	8000x6000 pixel	48.0 MP camera	JPG , RAW	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
2	10002	9216x6912 pixel	63.7 MP camera	JPG , RAW	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
3	10003	4000x3000 pixel	12.0 MP camera	JPG , HEIF	3840x2160 pixel	No	2026-07-27T06:46:02.81...	development	
4	10004	4000x3000 pixel	12.0 MP camera	JPG	3840x2160 pixel	No	2026-07-27T06:46:02.81...	development	
5	10005	8160x6144 pixel	50.1 MP camera	JPG	1920x1080 pixel	single LED	2026-07-27T06:46:02.81...	development	
6	10006	8000x6000 pixel	48.0 MP camera	JPG , HEIF	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
7	10007	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
8	10008	8160x6144 pixel	50.1 MP camera	JPG	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
9	10009	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	dual LED	2026-07-27T06:46:02.81...	development	
10	10010	8160x6144 pixel	50.1 MP camera	JPG , RAW	1920x1080 pixel	single LED	2026-07-27T06:46:02.81...	development	
11	10011	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	dual LED	2026-07-27T06:46:02.81...	development	
12	10012	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	dual LED	2026-07-27T06:46:02.81...	development	
13	10013	8192x6144 pixel	50.3 MP camera	JPG , RAW	3840x2160 pixel	dual LED	2026-07-27T06:46:02.81...	development	
14	10014	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
15	10015	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
16	10016	8160x6144 pixel	50.1 MP camera	JPG , RAW	3840x2160 pixel	single LED	2026-07-27T06:46:02.81...	development	
17	10017	8192x6144 pixel	50.3 MP camera	JPG , RAW	1920x1080 pixel	single LED	2026-07-27T06:46:02.81...	development	

Realizado correctamente (0 sec 460 ms)

Columnas 8 Filas 1,000

Carga cruda Lakehouse Bronze: se pueden denotar las carpetas con la data cruda y cargada desde on-premise, donde normalmente se realiza el Shortcut correspondiente desde el almacenamiento predefinido por la organización. (En caso de Pipeline no ocupamos un Shortcut).

Fabric

smart_device_analythcs_ws

Área de trabajo para el curso de Fabric en Proyecto 1

Development

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

Filtrar por palabra cl...

Filtro

	Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
	Copy Job		Carpeta	—	—	—	—	—	—	
	DataFlow_Gen2		Carpeta	—	—	—	—	—	—	
☐	Notebook	...	Carpeta	—	—	—	—	—	—	
	Pipeline		Carpeta	—	—	—	—	—	—	
	lh_bronze		Lakehouse	Bronze D...	JESUS LIBRA...	—	—	—	—	
	lh_bronze		Punto de con...	Bronze D...	JESUS LIBRA...	—	—	—	—	
	lh_gold		Lakehouse	Gold Data	JESUS LIBRA...	—	—	—	—	
	lh_gold		Punto de con...	Gold Data	JESUS LIBRA...	—	—	—	—	
	lh_silver		Lakehouse	Silver Data	JESUS LIBRA...	—	—	—	—	
	lh_silver		Punto de con...	Silver Data	JESUS LIBRA...	—	—	—	—	
	sm_smarth_device_lh		Modelo semá...	—	smart_device...	17/8/2026, 11:2...	N/D	—	—	
	StagingLakehouseForDataflows_202607310412:		Lakehouse	—	JESUS LIBRA...	—	—	—	—	

Creación de folders correspondientes a etapas y funciones para llevar orden y limpieza, apuntar a Notebook para ingresar. Donde se resguardan los Notebooks del proyecto en capas.

Fabric

smart_device_analythcs_ws

Área de trabajo para el curso de Fabric en Proyecto 1

Development

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

Filtrar por palabra cl...

Filtro

smart_device_analythcs_ws > Notebook

	Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
	email		Carpeta	—	—	—	—	—	—	
☐	includes	...	Carpeta	—	—	—	—	—	—	
	Ingestion		Carpeta	—	—	—	—	—	—	
	transformation		Carpeta	—	—	—	—	—	—	

Una vez dentro de Notebooks se tiene etapa de ingestión y transformación, Ingestión corresponde a bronze nuestro punto actual, transformación correspondiente a capa Silver, includes y email son información complementaria para uso dentro de los notebooks donde sin problema se pueden poner funciones, paths, etc para modularizar el Notebook.

Fabric

smart_device_analythcs_ws

Development

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

Filtrar por palabra cl...

Filtro

smart_device_analythcs_ws > Notebook > Ingestion

	Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
	01_ingestion_device		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
☐	02_ingestion_model		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	03_ingestion_category		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	04_ingestion_camera		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	05_ingestion_brand		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	06_ingestion_operationing_system		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	07_ingestion_connectivity		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	08_ingestion_display		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	
	09_ingestion_physical_specs		Cuaderno	Ingestion ...	JESUS LIBRA...	—	—	—	—	

Notebooks > Ingestión: Notebooks diseñados para cargar la data hacia el Lakehouse, preparándola para su respectiva limpieza y transformación en la capa Silver.

Fabric 01_ingestion_device Guardado

Inicio Editar Herramientas con IA Ejecutar Ver

Comentarios Historial Desarrollo Compartir

Explorador

- Elementos de datos Recursos
- Conexiones
- + Agregar elementos de datos
- OneLake
 - lh_silver
 - lh_bronze

- Ingestion del archivo "device.csv"

Paso 1: Leer el archivo CSV usando: "Dataframe Reader" de Spark

```
1 p_file_date = "2025-07-07"
2 p_environment = "Development"
```

[1] ✓ - Sesión lista en 10 s 994 ms. Command executed in 335 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:35:40 p.m. PySpark (Python)

```
1 %run configuration
```

[2] ✓ - Command executed in 22 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:35:42 p.m. PySpark (Python)

Warning: In reference configuration run, the default lakehouse of the main notebook will be the effective default lakehouse of the session during reference run. Recommend using absolute path to read/write lakehouse in the referenced notebooks.

```
1 %run common_function
```

[3] ✓ - Command executed in 23 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:35:44 p.m. PySpark (Python)

No conectado Autoguardado: activado 27 de 27 celdas seleccionadas

Creación de parámetros para incluir en el Notebook correspondiente y acceso Comando Mágico para incluir variable desde otro Notebook.

The screenshot displays the Microsoft Fabric web interface. The top navigation bar includes the 'Fabric' logo, a tab for '01_ingestion_device', and a search bar. The left sidebar contains navigation icons for 'Inicio', 'Áreas de trabajo', 'Copilot', 'Catalogo de OneLake', 'Supervisión', 'Tiempo real', 'Cargas de trabajo', 'clean_port', 'smart_device_analytic...', and 'Fabric'. The main workspace is divided into three panes. The left pane, titled 'Explorador', shows a tree view with 'Elementos de datos' and 'Recursos'. Under 'Elementos de datos', there are 'Conexiones' and a button to 'Agregar elementos de datos'. The 'OneLake' section is expanded, showing 'lh_silver' and 'lh_bronze'. The middle pane shows a notebook with two code cells. The first cell, labeled '[27]', contains a single line of Python code: `print(bronze_folder_path)`. Below the code, a status bar indicates the command was executed in 293 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 11:20:55 a.m. The second cell, labeled '[4]', contains two lines of Python code: `##Importar tipos de datos IMPORTANTE, antes de darle Struct o crear nuestros schemas para evitar errores de tipos` and `from pyspark.sql.types import StructType, StructField, IntegerType, DateType`. Below this code, a status bar indicates the command was executed in 256 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:35:46 p.m. The bottom pane shows a third code cell, labeled '[5]', containing a Python dictionary definition for a schema: `device_schema = StructType(fields= [StructField("DeviceID", IntegerType(), False), StructField("BrandID", IntegerType(), True), StructField("ModelID", IntegerType(), True), StructField("DisplayID", IntegerType(), True), StructField("CameraID", IntegerType(), True), StructField("ConnectivityID", IntegerType(), True), StructField("OSID", IntegerType(), True), StructField("PhysicalSpecID", IntegerType(), True), StructField("ReleasedYear", IntegerType(), True), StructField("ReleasedAnnounced", DateType(), True)])`. Below this code, a status bar indicates the command was executed in 254 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:35:48 p.m. The bottom right corner of the interface shows '27 de 27 celdas seleccionadas'.

Verificación de funcionamiento adecuado de la ruta guardada en otro notebook y adaptado con Comando Mágico, Importación de librerías correspondientes, uso de StructField, StructType, y tipos de datos que se manejan en la tabla, para la creación de un esquema explícito alineado al tipo de data del dataset, buscando estandarizar los tipos de datos antes de proceder a utilizarlos.

The screenshot displays the Microsoft Fabric workspace interface. The top navigation bar includes the 'Inicio' (Home) tab, a search bar, and various utility icons. The left sidebar contains navigation options like 'Inicio', 'Áreas de trabajo', 'Copilot', 'Catálogo de OneLake', 'Supervisión', 'Tiempo real', 'Cargas de trabajo', 'smart_device_analytic...', and 'Real_Time_Intelligence'. The main workspace is divided into three panes: 'Explorador' (Explorer) on the left, a code editor in the center, and a console/output pane at the bottom.

The 'Explorador' pane shows a tree view of data elements, including 'Elementos de datos' and 'Conexiones'. Under 'Conexiones', there is a button to 'Agregar elementos de datos'. The 'OneLake' section lists two data lakes: 'lh_silver' and 'lh_bronze'.

The code editor displays a PySpark script (Python) for reading a CSV file. The script includes comments in Spanish and uses a Try-Except block for error handling. The code reads a CSV file from a specified path, validates the schema, and prints messages indicating the success or failure of the operation.

```
1  ## Crear nuestros propios Schemas para millones de registros
2  ## .Inferschema para delegarle a spark la lectura adecuada de schemas solo con poco registros (sino con nuestro propio schema)
3  ## .csv para dar lectura al archivo csv
4  try:
5      print("Inicio de procedimiento de lectura de archivo: ")
6      device_df = spark.read\
7          .option("header", True)\
8          .schema(device_schema) \
9          .csv(f"{bronze_folder_path}/{p_file_date}/device.csv")
10
11      if device_df.isEmpty():##Estructura de validacion en caso de error en ruta
12          print("Advertencia Dataframe vacio")
13  except Exception as e:
14      print(f"Error critico detectado: , {str(e)}")
15      raise e
16  else:
17      print("Lectura de Dataframe correcta")
18  finally:
19      print("Procedimiento de lectura finalizado")
20
21
```

The console pane shows the execution results of the script. The first command executed successfully, printing the start of the procedure. The second command executed successfully, printing the end of the procedure.

```
[6] ✓ - Command executed in 1 s 309 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:35:53 p.m. PySpark (Python)
...
[31] ✓ - Command executed in 10 s 168 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 11:21:32 a.m. PySpark (Python)
```

The bottom status bar indicates 'No conectado' (Not connected) and 'Autoguardado: activado' (Autosave: activated). The bottom right corner shows '9 de 27 celdas seleccionadas' (9 of 27 cells selected).

Lectura de archivo CSV con manejo de errores (Try-Except): Validación de archivos vacíos mediante raise para detener la ejecución y notificar incidencias. Lectura con Spark aplicando cabeceras (*headers*) y acoplando el esquema explícito predefinido junto con su ruta base.

The screenshot shows the Microsoft Fabric interface. The top bar includes the 'Fabric' logo, a tab for '01_ingestion_device', and a search bar. The left sidebar contains navigation icons for 'Inicio', 'Áreas de trabajo', 'Copilot', 'Catálogo de OneLake', 'Supervisión', 'Tiempo real', 'Cargas de trabajo', and 'clean_port'. The main area displays a code cell with the following PySpark code:

```
1 ##printSchema() para poder analizar y visualizar las tipologias columnares.
2 ##Nota: DeviceID por defecto aparece nullable=true por defecto pero por debajo esta denegado nullable al ser
3 ##una pk no no es permitido manerj este apartado de forma null
4 device_df.printSchema()
```

The output of the command is shown below the code:

```
root
 |-- DeviceID: integer (nullable = true)
 |-- BrandID: integer (nullable = true)
 |-- ModelID: integer (nullable = true)
 |-- DisplayID: integer (nullable = true)
 |-- CameraID: integer (nullable = true)
 |-- ConnectivityID: integer (nullable = true)
 |-- OSID: integer (nullable = true)
 |-- PhysicalSpecID: integer (nullable = true)
 |-- ReleasedYear: integer (nullable = true)
 |-- ReleasedAnnounced: date (nullable = true)
```

Una vez leído el archivo verificar los tipos de datos correspondientes al esquema como buena práctica, control y validación.

The screenshot displays the Microsoft Fabric workspace interface. The top navigation bar includes the 'Fabric' logo, a document titled '01_ingestion_device', and a search bar. The left sidebar shows the 'Explorador' (Explorer) pane with 'Elementos de datos' (Data Elements) and 'Recursos' (Resources) tabs. Under 'OneLake', there are folders for 'lh_silver' and 'lh_bronze'. The main workspace area is titled 'Paso 2: Seleccionar las columnas requeridas' (Step 2: Select the required columns). It contains three code blocks, each with a PySpark script and a status message indicating successful execution.

Explorador

- Elementos de datos
- Recursos
- Conexiones
- + Agregar elementos de datos
- OneLake
 - lh_silver
 - lh_bronze

Paso 2: Seleccionar las columnas requeridas

```
1 ##Importacion de col para seleccionar solo las columnas necesarias
2 from pyspark.sql.functions import col
```

[9] ✓ - Command executed in 291 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:36:29 p.m. PySpark (Python)

```
1 ##col permitira unicamente seleccionar columnas a utilizar o en su defecto la utilizacion de drop a la
2 ##inversa de eliminar las columnas que no ocupemos
3 device_selected_df = device_df.select(col("DeviceID"),col("BrandID"),col("ModelID"),col("DisplayID"),col("CameraID")
4 .....col("ConnectivityID"),col("OSID"),col("PhysicalSpecID"),col("ReleasedAnnounced"))
```

[10] ✓ - Command executed in 254 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:36:32 p.m. PySpark (Python)

```
1 ##Verificacion de correcta aplicacion de la seleccion adecuada de las columnas idoneas
2 display(device_selected_df.limit(10))
3
```

[37] ✓ - Command executed in 2 s 163 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 11:22:51 a.m. PySpark (Python)

Importación de librería correspondiente de “col” para la selección de solo las columnas requeridas en combinación con el uso de “select”, utilización de “display” para verificación de cambios limitado a 10 registros para asegurar poco procesamiento pero permitiéndonos revisar los cambios.

Fabric 01_ingestion_device Guardado

Inicio Editar Herramientas con IA Ejecutar Ver

Ejecutar todo Conectar PySpark (Python) Entorno Valor predeterminado ... Data Wrangler Copilot

Explorador

Elementos de datos Recursos

Conexiones

+ Agregar elementos de datos

OneLake

- lh_silver
- lh_bronze

****Paso 3: Cambiar el nombre de las columnas segun lo requerido

```
1 from pyspark.sql.functions import current_date
```

```
1 ##Utilizacion de withColumnRenamed para modificar a nombres basados en snake_case por tema de preferencia
2 ##Podria abordarse .withColumn para agregar las columnas "Ingestion_date" y "Environment" pero por temas de practicidad
3 ##Se usaron aparte solo agregamos
4 device_renamed_df = device_selected_df\
5     .withColumnRenamed('DeviceId', 'device_id') \
6     .withColumnRenamed('BrandId', 'brand_id') \
7     .withColumnRenamed('ModelId', 'model_id') \
8     .withColumnRenamed('DisplayId', 'display_id') \
9     .withColumnRenamed('CameraId', 'camera_id') \
10    .withColumnRenamed('ConnectivityId', 'connectivity_id') \
11    .withColumnRenamed('OSID', 'os_id') \
12    .withColumnRenamed('PhysicalSpecId', 'physical_spec_id') \
13    .withColumnRenamed('ReleasedAnnounced', 'released_announced') \
14    .withColumn('Ingestion_date', current_date())
```

[11] ✓ - Command executed in 271 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:36:39 p.m.

Estandarización de nombres de columnas a formato snake_case mediante `.withColumnRenamed` y almacenamiento en un DataFrame temporal, asegurando formalidad, compatibilidad entre herramientas y facilidad de uso (con flexibilidad para adaptarse a futuras necesidades).

Fabric 01_ingestion_device

Buscar

Pruebas activ... Queda 12 días

Inicio Editar Herramientas con IA Ejecutar Ver

Ejecutar todo Conectar PySpark (Python) Entorno Valor predeterminado ... Data Wrangler Copilot

Explorador

Elementos de datos Recursos

Conexiones

+ Agregar elementos de datos

OneLake

- lh_silver
- lh_bronze

```
1 ##Verificación de renombramientos
2 display(device_renamed_df.limit(100))
```

[39] ✓ - Command executed in 2 s 705 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 11:23:19 a.m.

PySpark (Python)

Table + Nuevo gráfico

9 columnas, 1000 filas

Descargar Filtar por palabra

Vista de tabla

	brand_id	model_id	display_id	camera_id	connectivity_id	os_id	physical_spec_id	ReleasedAnnounced
1	101	1001	10001	101	1	1		2025-03-19
1	102	1001	10001	101	1	1		2025-03-19
1	103	1001	10001	102	1	1		2025-03-19
1	104	1001	10001	101	1	2		2025-03-19
1	105	1001	10001	102	1	2		2025-03-19
1	106	1002	10002	101	2	3		2024-05-07
1	107	1002	10002	101	2	3		2024-05-07
1	108	1002	10002	101	2	3		2024-05-07
1	109	1002	10002	101	2	4		2024-05-07
1	110	1002	10002	102	2	4		2024-05-07
1	111	1002	10002	101	2	4		2024-05-07
1	112	1002	10002	101	2	4		2024-05-07
1	113	1001	10001	101	1	2		2025-03-19
2	114	1003	10003	103	3	5		2025-03-04
2	115	1003	10003	103	3	6		2025-03-04
2	116	1003	10003	103	3	7		2025-03-04
2	117	1003	10003	103	3	8		2025-03-04

Verificación de los cambios de nombre o estandarización de estos, siguiendo la premisa de limitación para bajo procesamiento solo para poder cerciorarse de cambios adecuados.

The screenshot displays the Microsoft Fabric notebook environment. The top navigation bar includes the 'Fabric' logo, a tab for '01_ingestion_device', and a 'Guardado' (Saved) status. A search bar is present on the right. The left sidebar shows the 'Explorador' (Explorer) pane with 'Elementos de datos' (Data Elements) and 'Recursos' (Resources) sections. Under 'Elementos de datos', there are 'Conexiones' (Connections) and a list of OneLake catalogs including 'lh_silver' and 'lh_bronze'. The main workspace shows a notebook with a title 'Paso 4: Agregar columnas a 'Environment' y file_date'. The notebook content includes two code blocks. The first code block imports the 'lit' function from 'pyspark.sql.functions'. The second code block uses 'add_ingestion_date' to add 'environment' and 'file_date' columns to a DataFrame. Both code blocks show successful execution results with timestamps and user information.

Explorador

- Elementos de datos
- Recursos
- Conexiones
- + Agregar elementos de datos
- OneLake
 - lh_silver
 - lh_bronze

Paso 4: Agregar columnas a 'Environment' y file_date

```
1 ##Para usar el current_timestamp que refiere a la hora del sistema actual hay que importarle
2 from pyspark.sql.functions import lit
```

[13] ✓ - Command executed in 385 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:36:51 p.m. PySpark (Python)

```
1 ##Agregar columnas una por una, el current_time stamp ya esta en la funcion add_ingestion_date
2 device_final_df = add_ingestion_date(device_renamed_df)\
3     .withColumn('environment', lit(p_environment))\
4     .withColumn("file_date", lit(p_file_date))
```

[21] ✓ - Command executed in 277 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:39:05 p.m. PySpark (Python)

Importación de “lit” para agregar un par de columnas base, a través de “lit” permite poner personalización en nombres de las columnas nuevas, como “environment” y “file_date”, donde environment juega un papel de categorización y file_date como base para parámetro en pipeline, cabe mencionar que se apuntó a las variables globales creadas al inicio del notebook.

Fabric 01_ingestion_device

Inicio Editar Herramientas con IA Ejecutar Ver

Ejecutar todo Conectar PySpark (Python) Entorno Valor predeterminado ... Data Wrangler Copilot

Explorador

Elementos de datos Recursos

Conexiones

+ Agregar elementos de datos

OneLake

- lh_silver
- lh_bronze

```
1 ##Verifican la agregacion correcta de las columnas correspondientes
2 display(device_final_df)
```

✓ - Command executed in 1 s 341 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:39:09 p.m.

Table + Nuevo gráfico

Vista de tabla

Descargar Filtrar por palabra

123	camera_id	123	connectivity_id	123	os_id	123	physical_spec_id	released_announced	Ingestion_date	ABC	environment	ABC	file_date
10001		102		1		2		2025-03-19	2026-08-05 19:39:00...		Development		2025-07-07
10002		101		2		3		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10002		101		2		3		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10002		101		2		3		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10002		101		2		4		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10002		102		2		4		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10002		101		2		4		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10002		101		2		4		2024-05-07	2026-08-05 19:39:00...		Development		2025-07-07
10001		101		1		2		2025-03-19	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		5		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		6		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		7		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		8		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		9		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		10		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		103		3		11		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07
10003		104		3		12		2025-03-04	2026-08-05 19:39:00...		Development		2025-07-07

No conectado Autoguardado: activado

23 de 27 celdas seleccionadas

Verificación de agregación de nuevas columnas clave “environment” y “file_date”

The screenshot displays the Databricks Fabric IDE interface. The left sidebar shows the workspace structure with '01_ingestion_device' selected. The main area is titled 'Paso 5: Escribir datos en el "LakeHouse" en formato Delta'. It contains three code blocks:

- Block 1 (Spark SQL):** A command to drop a table if it exists.

```
1 %%sql
2 -- Verificacion de si la tabla existe si existe la elina para darle cabida a la tabla limpia en dicho werehouse y si no la crea
3 DROP TABLE IF EXISTS device
```

Execution status: [23] ✓ - Command executed in 2 s 26 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:39:20 p.m.
- Block 2 (PySpark):** A command to write data to a table in Delta format with overwrite mode.

```
1 ##Transformacion a formato delta
2 ##Sobre escritura en este caso
3 ##Guardado como tabla para indexacion con SQL
4 device_final_df.write.format("delta").mode("overwrite").saveAsTable("device")
```

Execution status: [24] ✓ - Command executed in 4 s 320 ms by JESUS LIBRADO CHAVEZ HUERTA on 5/8/2026, 12:39:24 p.m.
- Block 3 (Spark SQL):** A command to select all data from the 'device' table to verify the load.

```
1 %%sql
2 --Verificacion de carga correcta y consulta con SQL
3 SELECT *
4 FROM device
5 LIMIT 10;
6 --finalmente verificar que se encuentre todo en formato DELTA
```

The bottom status bar indicates 'No conectado' and 'Autoguardado: activado'. The bottom right corner shows '23 de 27 celdas seleccionadas'.

Una vez realizados los cambios correspondientes, verificación de existencia de la tabla a través de T-SQL, guardado de tabla en formato Delta apuntando a Lakehouse Silver con un modo de Overwrite y un guardado tipo tabla, una vez guardado verificación de carga de tabla mediante T-SQL.

Fabric lh_silver X

Inicio Vistas de lago materializadas

Obtener datos Nueva consulta de SQL de Spark Nuevo modelo semántico Agregar al agente de datos Administrar la seguridad de OneLake Actualizar todas las variables

Explorador

Agregar almacenes de lago de datos

lh_silver

Tables

dbo

brand

camera

category

connectivity

device

display

model

op_system

physical_specs

Files

brand

Vista de tabla

Mostrando 944 filas

	123	brand_id	ABC brand	ABC hardware_designer	ABC manufacturer	ingestion_time	ABC environment	ABC file_date
1	1		Google	Google	Foxconn	2026-08-11T05:51:56.749Z	Development	2025-07-14
2	2		Apple	Apple	Foxconn	2026-08-11T05:51:56.749Z	Development	2025-07-14
3	3		Samsung			2026-08-11T05:51:56.749Z	Development	2025-07-14
4	4		Samsung	Samsung Electronics		2026-08-11T05:51:56.749Z	Development	2025-07-14
5	5		Samsung	Samsung Electronics	Samsung Electronics	2026-08-11T05:51:56.749Z	Development	2025-07-14
6	6		Motorola	Lenovo		2026-08-11T05:51:56.749Z	Development	2025-07-14
7	7		Motorola	Lenovo	Lenovo	2026-08-11T05:51:56.749Z	Development	2025-07-14
8	8		NEC	NEC	NEC	2026-08-11T05:51:56.749Z	Development	2025-07-14
9	9		NEC			2026-08-11T05:51:56.749Z	Development	2025-07-14
10	10		Nothing	Nothing Technology Limited	BBK Electronics	2026-08-11T05:51:56.749Z	Development	2025-07-14
11	11		Asus	ASUSTeK Computer	ASUSTeK Computer	2026-08-11T05:51:56.749Z	Development	2025-07-14
12	12		Asus	ASUSTeK Computer	Foxconn	2026-08-11T05:51:56.749Z	Development	2025-07-14
13	13		Oppo	Oppo	BBK Electronics	2026-08-11T05:51:56.749Z	Development	2025-07-14
14	14		OnePlus	Oppo	BBK Electronics	2026-08-11T05:51:56.749Z	Development	2025-07-14
15	15		OnePlus	OnePlus	BBK Electronics	2026-08-11T05:51:56.749Z	Development	2025-07-14
16	16		OnePlus	BBK Electronics	BBK Electronics	2026-08-11T05:51:56.749Z	Development	2025-07-14
17	17		OnePlus	BBK Electronics		2026-08-11T05:51:56.749Z	Development	2025-07-14

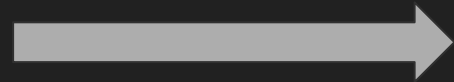
Realizado correctamente (1 sec 563 ms)

Columnas 7 Filas 944

Verificación e identificación visual en Lakehouse Silver para asegurar la carga correcta.

Notebook Configuration donde se almacena en una variable la ruta de bronze y una para la ruta gold para la carga de archivo a través del comando mágico.

- Se realizaron nueve procesos de ingesta bajo parámetros y metodología de trabajo similar a la del ejemplo anterior, se tomó solo una como ejemplificación y muestra para no extender en demasía el portafolio.
- Paso consiguiente limpieza en Lakehouse Silver se realizaron cuatro limpiezas a través de Dataflow el cual se tomó como referencia el no usarle en archivos mayores a 2gb de volumen, si pesara mas de lo mencionado habría que llevarlo a un Notebook. Y si aun así por disposición empresarial se solicitará trabajar archivos pequeños en Notebooks utilización (OPTIMIZE + V-order) para garantizar el óptimo funcionamiento de archivos pequeños mediante la optimización requerida para que el motor de Spark no se sature al tener pocos registros.



Fabric

smart_device_analythcs_ws

Area de trabajo para el curso de Fabric en Proyecto 1

Ver canalización de implementación Crear aplicación Administrar acceso Configuración del área de trabajo

+ Nuevo elemento Nueva carpeta Importar Migrar

Filtrar por palabra cl... Filtro

Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
Copy Job		Carpeta	—	—	—	—	—	—	
DataFlow_Gen2		Carpeta	—	—	—	—	—	—	
Notebook		Carpeta	—	—	—	—	—	—	
Pipeline		Carpeta	—	—	—	—	—	—	
lh_bronze		Lakehouse	Bronze D...	JESUS LIBRA...	—	—	—	—	
lh_bronze		Punto de con...	Bronze D...	JESUS LIBRA...	—	—	—	—	
lh_gold		Lakehouse	Gold Data	JESUS LIBRA...	—	—	—	—	
lh_gold		Punto de con...	Gold Data	JESUS LIBRA...	—	—	—	—	
lh_silver		Lakehouse	Silver Data	JESUS LIBRA...	—	—	—	—	
lh_silver		Punto de con...	Silver Data	JESUS LIBRA...	—	—	—	—	
sm_smarth_device_lh		Modelo semá...	—	smart_device...	17/8/2026, 11:2...	N/D	—	—	
StagingLakehouseForDataflows_202607310412:		Lakehouse	—	JESUS LIBRA...	—	—	—	—	

Creación de folder DataFlow para llevar orden, secuencial y por herramienta.

Fabric

smart_device_analythcs_ws

Development

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

smart_device_analythcs_ws > DataFlow_Gen2

	Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
	01_transformation_table_brand	✓	Flujo de dato...	Transfer...	JESUS LIBRA...	11/8/2026, 15:44...	—	—	—	
	02_transformation_table_camera	✓	Flujo de dato...	Transfer...	JESUS LIBRA...	11/8/2026, 15:45...	—	—	—	
	03_transformation_table_connectivity	✓	Flujo de dato...	Transfer...	JESUS LIBRA...	11/8/2026, 15:47...	—	—	—	
	04_transformation_table_op_system	✓	Flujo de dato...	Transfer...	JESUS LIBRA...	11/8/2026, 15:48...	—	—	—	

Archivo de Dataflow utilizado para la limpieza en la capa Silver (aplicado bajo la consideración de procesar archivos menores a 2 GB).

Fabric 01_transformation_table_brand X

Power Query

Inicio Transformación Agregar columna Ver Ayuda

Table.RemoveColumns("#Valor reemplazado 1", {"environment"})

Consultas [1]

brand

	1-3 brand_id	1-3 brand	1-3 hardware_designer	1-3 manufacturer	1-3 ingestion_time	1-3 file_date
1	1	Google	Google	Foxconn	11/8/2026, 5:51:56	2025-07-14
2	2	Apple	Apple	Foxconn	11/8/2026, 5:51:56	2025-07-14
3	3	Samsung	Unknown	Unknown	11/8/2026, 5:51:56	2025-07-14
4	4	Samsung	Samsung Electronics	Unknown	11/8/2026, 5:51:56	2025-07-14
5	5	Samsung	Samsung Electronics	Samsung Electronics	11/8/2026, 5:51:56	2025-07-14
6	6	Motorola	Lenovo	Unknown	11/8/2026, 5:51:56	2025-07-14
7	7	Motorola	Lenovo	Lenovo	11/8/2026, 5:51:56	2025-07-14
8	8	NEC	NEC	NEC	11/8/2026, 5:51:56	2025-07-14
9	9	NEC	Unknown	Unknown	11/8/2026, 5:51:56	2025-07-14
10	10	Nothing	Nothing Technology Limited	BBK Electronics	11/8/2026, 5:51:56	2025-07-14
11	11	Asus	ASUSTeK Computer	ASUSTeK Computer	11/8/2026, 5:51:56	2025-07-14
12	12	Asus	ASUSTeK Computer	Foxconn	11/8/2026, 5:51:56	2025-07-14
13	13	Oppo	Oppo	BBK Electronics	11/8/2026, 5:51:56	2025-07-14
14	14	OnePlus	Oppo	BBK Electronics	11/8/2026, 5:51:56	2025-07-14
15	15	OnePlus	OnePlus	BBK Electronics	11/8/2026, 5:51:56	2025-07-14
16	16	OnePlus	BBK Electronics	BBK Electronics	11/8/2026, 5:51:56	2025-07-14
17	17	OnePlus	BBK Electronics	Unknown	11/8/2026, 5:51:56	2025-07-14
18	18	Google	Unknown	Sunwoda Electronic	11/8/2026, 5:51:56	2025-07-14
19	19	Google	Unknown	Shenzhen Highpower Technology	11/8/2026, 5:51:56	2025-07-14
20	20	Google	Compal Electronics	Compal Electronics	11/8/2026, 5:51:56	2025-07-14
21	21	Xiaomi	Xiaomi	Foxconn	11/8/2026, 5:51:56	2025-07-14
22	22	Xiaomi	Xiaomi	Unknown	11/8/2026, 5:51:56	2025-07-14
23	23	Fujitsu	Fujitsu	Fujitsu	11/8/2026, 5:51:56	2025-07-14
24	24	Oppo	Oppo	Unknown	11/8/2026, 5:51:56	2025-07-14
25	25	BBK	BBK Electronics	Unknown	11/8/2026, 5:51:56	2025-07-14

Configuración de cor >

Propiedades

Nombre

brand

Pasos aplicados

- Origen
- Navegación
- Navegación...
- Navegación...
- Valor reem...
- Valor reem...
- Columnas ...

Destino de datos

Lakehouse

Se ha completado en (13.30 s) Columnas: 6 Filas: 99 más Agregar destino predeterminado...

Paso 000

Modificación de 2 columnas en valor y eliminación de 1 columna no útil para la finalidad.(Se pueden realizar muchos tipos de modificaciones pero dependen del estado de proveniencia del Dataset).

Fabric

smart_device_analythcs_ws

Área de trabajo para el curso de Fabric en Proyecto 1

Development

Ver canalización de implementación Crear aplicación Administrar acceso Configuración del área de trabajo

+ Nuevo elemento Nueva carpeta Importar Migrar

Filtrar por palabra cl... Filtro

smart_device_analythcs_ws > Notebook

	Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
	email		Carpeta	—	—	—	—	—	—	
	includes		Carpeta	—	—	—	—	—	—	
	Ingestion		Carpeta	—	—	—	—	—	—	
☐	transformation	...	Carpeta	—	—	—	—	—	—	

Se pasa a Limpieza mediante Notebook, Se crea carpeta Transformation dentro de la carpeta Notebooks para mantener una Jerarquía de procesos y mantener orden.

Fabric

smart_device_analythcs_ws

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

smart_device_analythcs_ws > Notebook > transformation

Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
01_transformation_table_category		Cuaderno	Transfer...	JESUS LIBRA...	—	—	—	—	
02_transformation_table_device		Cuaderno	Transfer...	JESUS LIBRA...	—	—	—	—	
03_transformation_table_model		Cuaderno	Transfer...	JESUS LIBRA...	—	—	—	—	
04_transformation_table_display		Cuaderno	Transfer...	JESUS LIBRA...	—	—	—	—	
05_transformation_table_physical_specs		Cuaderno	Transfer...	JESUS LIBRA...	—	—	—	—	
06_create_dim_time		Cuaderno	Silver Data	JESUS LIBRA...	—	—	—	—	

Archivo de Notebook utilizado para la limpieza en la capa Silver (aplicado bajo la consideración de procesar archivos mayores a 2 GB).

Fabric 03_transformation_table_model Guardado Buscar Pruebas activ... Queda 12 días

Inicio Editar Herramientas con IA Ejecutar Ver Comentarios Historial Desarrollo Compartir

Ejecutar todo Conectar PySpark (Python) Entorno Valor predeterminado ... Data Wrangler Copilot

Explorador

Elementos de datos Recursos

Conexiones

+ Agregar elementos de datos

OneLake

- lh_gold
- lh_silver

Menú de salida

```
1 ##Inyeccion de dependencias
2 %run configuration
```

[8] ✓ - Command executed in 1 s 194 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:54:56 p.m. PySpark (Python)

Warning: In reference configuration run, the default lakehouse of the main notebook will be the effective default lakehouse of the session during reference run. Recommend using absolute path to read/write lakehouse in the referenced notebooks.

```
1 ##Carga de Df desde Lakehouse silver schema dbo tabla model
2 model_df = spark.sql("SELECT * FROM lh_silver.dbo.model")
```

[2] ✓ - Sesión lista en 9 s 774 ms. Command executed in 9 s 158 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:44:24 p.m. PySpark (Python)

```
1 ##Verificacion de carga exitosa.
2 ##Podemos solo ejecutar un limit(100) por ejemplo para evitar se saturate por temas de praticidad se ejecuto asi
3 display(model_df)
```

[2] ✓ - Command executed in 7 s 403 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:44:32 p.m. PySpark (Python)

Table + Nuevo gráfico 7 columnas, 1000 filas

Vista de tabla

	model_id	category_id	ABC model	ABC codename	ingestion_date	ABC environment	ABC file_date
1	101	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	Development	2025-07-07
2	102	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	Development	2025-07-07

8 de 10 celdas seleccionadas

Inyección de dependencias con comando mágico, carga de Dataset desde Lakehouse Silver mediante uso de T-SQL, y verificación de carga correcta.

The screenshot displays the Microsoft Fabric Data Studio environment. The top navigation bar includes tabs for 'Inicio', 'Editar', 'Herramientas con IA', 'Ejecutar', and 'Ver'. Below this is a toolbar with icons for saving, executing, connecting, and other functions. The left sidebar, titled 'Explorador', shows a tree view of data elements and connections, including 'OneLake' and specific datasets like 'lh_gold' and 'lh_silver'. The main workspace contains a PySpark notebook with three code cells:

```
1 ##Conteo de registro mediante count y antes de ejecucion de merge para carga incremental
2 model_df.count()
```

[3] ✓ - Command executed in 1 s 407 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:44:49 p.m. PySpark (Python)

10000

```
1 ##Verificacion de tipologias nuevamente
2 model_df.printSchema()
```

[4] ✓ - Command executed in 298 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:45:04 p.m. PySpark (Python)

...

```
root
 |-- model_id: integer (nullable = true)
 |-- category_id: integer (nullable = true)
 |-- model: string (nullable = true)
 |-- codename: string (nullable = true)
 |-- ingestion_date: timestamp (nullable = true)
 |-- environment: string (nullable = true)
 |-- file_date: string (nullable = true)
```

```
1 ##Importacion col para eliminar columna no utilizable de momento
2 from pyspark.sql.functions import col
```

[5] ✓ - Command executed in 321 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:45:57 p.m. PySpark (Python)

The bottom status bar indicates 'No conectado' and 'Autoguardado: activado'. The bottom right corner shows '8 de 10 celdas seleccionadas'.

Conteo de elementos mediante “Count” para verificación de total, revisión de tipologías correctas aplicadas en el Esquema del Dataset a través de “.printSchema()” e importación de librería “col” para eliminar columna no utilizable.

Screenshot of the Microsoft Fabric interface showing a PySpark notebook execution. The interface includes a sidebar with navigation options like Inicio, Áreas de trabajo, Copilot, and Supervisión. The main workspace displays two code blocks and their execution results.

Code Block 1:

```
1 ##Creacion de nuevo Df con la eliminacion de la columna 'environment'
2 model_final_df = model_df.drop(col('environment'))
3
```

Execution Result 1:

[6] ✓ - Command executed in 291 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:47:24 p.m. PySpark (Python)

Code Block 2:

```
1 ##Verificacion de eliminacion correcta de la columan 'environment'
2 display(model_final_df.limit(10))
```

Execution Result 2:

[7] ✓ - Command executed in 13 s 983 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 9:47:56 p.m. PySpark (Python)

Table View:

Vista de tabla

	123_model_id	123_category_id	ABC_model	ABC_codename	📅_ingestion_date	ABC_file_date
1	101	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	2025-07-07
2	102	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	2025-07-07
3	103	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	2025-07-07
4	104	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	2025-07-07
5	105	1	Pixel 9a 5G ...	Google Tegu	2026-08-05 19:41:...	2025-07-07
6	106	1	Pixel 8a 5G ...	Google Akita	2026-08-05 19:41:...	2025-07-07
7	107	1	Pixel 8a 5G ...	Google Akita	2026-08-05 19:41:...	2025-07-07
8	108	1	Pixel 8a 5G ...	Google Akita	2026-08-05 19:41:...	2025-07-07

8 de 10 celdas seleccionadas

Utilización de “col” en conjunción con “.drop” para eliminación de columna que no aporta valor al análisis, verificación con “display” para corroboración de cambios aplicados, limitado a 10 para evitar saturación pero suficiente para denotar las modificaciones.

The screenshot displays the Microsoft Fabric Data Wrangler interface. The top navigation bar includes tabs for 'Inicio', 'Editar', 'Herramientas con IA', 'Ejecutar', and 'Ver'. A search bar is present on the right. The left sidebar shows the 'Explorador' (Explorer) pane with 'Elementos de datos' (Data Elements) and 'Recursos' (Resources). Under 'OneLake', there are folders for 'lh_gold' and 'lh_silver'. The main workspace shows a PySpark script with the following code:

```
1  ##Merge
2  ##Importacion de librerías correspondientes
3  from delta.tables import *
4  from pyspark.sql.functions import col
5
6
7  ##Validacion de existencia de tablas
8  if spark.catalog.tableExists("lh_gold.dbo.fact_model"):
9
10     ##Introducir la tabla de origen y de destino en este caso solo destino porque el origen es el device_final_df
11     deltaTableModel = DeltaTable.forPath(spark, gold_folder_path)## se puede apuntar por nombre en otro tipo de estructura
12     ##Establecer alias para el tabla de origen y comparar datasets
13     ##Con whenMatchedUpdate le decimos que cuando la coincidencia exista actualize esa particion
14     deltaTableModel.alias('tgt') \
15     .merge(
16         model_final_df.alias('src'),
17         (col('tgt.model_id') == col('src.model_id')) & (col('tgt.file_date') == col('src.file_date'))
18     )\
19     .whenMatchedUpdateAll()\
20     .whenNotMatchedInsertAll()\
21     .execute()
22 else:
23     model_final_df.write.format('delta').mode('overwrite').partitionBy('file_date').saveAsTable('fact_model')
24     ##Cuando es carga incremental se debe establecer el partitionBy
25     ##por la columna para particion en este caso file_date y se ejecuta un merge.
26
27     ##Cuando es una carga completa no es necesario un merge y utilizamos overwrite
28
```

At the bottom of the script editor, a status bar indicates: [9] ✓ - Command executed in 11 s 90 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 10:32:15 p.m. The bottom right corner shows 'PySpark (Python)' and '8 de 10 celdas seleccionadas'.

Importación de librerías correspondientes, Aplicación de Merge para carga incremental, verificación con condicional para comprobación de existencia de tabla mediante ruta, carga de tabla tipo Delta y asignación de alias en origen y destino, especificación de columnas para hacer match, ejecución de update con .WhenMatchedUpdateAll() y con WhenNotMatchedUpdateAll() y Ejecución y si no aplicar guardado en formato Delta con Overwrite apuntando a PartitionBy por el campo “file_date” guardando como Tabla de Factos, Ademas de apuntar el guardado en Lakehouse Gold.

The screenshot displays the Microsoft Fabric user interface. At the top, the breadcrumb navigation shows 'Fabric' followed by a tab for '03_transformation_table_model'. The top right corner contains status information: 'Pruebas activ...' and 'Queda 12 días'. Below this is a toolbar with buttons for 'Comentarios', 'Historial', 'Desarrollo', and 'Compartir'. The main interface is divided into three sections:

- Left Panel (Explorador):** Contains 'Elementos de datos' and 'Recursos'. Under 'Elementos de datos', there is a 'Conexiones' section with a search bar and a '+ Agregar elementos de datos' button. Below this, under 'OneLake', there are two items: 'lh_gold' and 'lh_silver'.
- Center Panel:** Displays a PySpark (Python) code editor. The code includes comments in Spanish and a Delta table write operation. Line 23 shows a Delta table write with 'overwrite' and 'partitionBy' on 'file_date'. Lines 24-26 explain that for incremental loads, a partitionBy is needed and a merge is executed. Lines 27-28 explain that for full loads, a merge is not necessary and 'overwrite' is used. Below the code, a status bar indicates '[9] ✓ - Command executed in 11 s 90 ms by JESUS LIBRADO CHAVEZ HUERTA on 6/8/2026, 10:32:15 p.m.'.
- Right Panel:** Displays a SQL query in a 'Spark SQL' editor. The query is a SELECT statement with a COUNT(1) aggregation, grouped by file_date.

Una vez aplicado el merge incremental y guardado como tabla Delta (o añadido a la tabla correspondiente), se realiza la verificación y el conteo de elementos para garantizar la integridad de los datos y evitar pérdidas de información.

Fabric lh_gold x

Buscar

Pruebas activ...
Queda 11 días

Inicio

Vistas de lago materializadas

Obtener datos Nueva consulta de SQL de Spark Nuevo modelo semántico Agregar al agente de datos Administrar la seguridad de OneLake Actualizar todas las variables

Explorador

Agregar almacenes de lago de datos

lh_gold

Tables

dbo

dim_brand

dim_camera

dim_category

dim_connectivity

dim_displayy

dim_op_system

dim_physical_sp...

dim_time

fact_device

fact_model

Files

dim_brand

Vista de tabla

Mostrando 944 filas

	123	brand_id	ABC brand	ABC hardware_designer	ABC manufacturer	ingestion_time	ABC file_date
1	31		Huawei	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
2	74		Huawei	Huawei	Global Components	2026-08-11T05:51:56.749Z	2025-07-14
3	145		GFive	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
4	366		SoftBank	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
5	426		KDDI	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
6	512		Hewlett-Packard	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
7	616		NTT DoCoMo	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
8	624		T-Mobile	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
9	639		Emobile	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
10	642		Orange	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
11	648		Sprint	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
12	663		Cincinnati Bell	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
13	665		Optimus	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
14	667		WellcoM	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
15	676		Vodafone	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
16	690		Red Bull Mobile	Huawei	Huawei	2026-08-11T05:51:56.749Z	2025-07-14
17	18		Google	Unknown	Sunwoda Electronic	2026-08-11T05:51:56.749Z	2025-07-14

Realizado correctamente (2 sec 144 ms)

Columnas 6 Filas 944

Verificación visual de la carga de datos en Lakehouse Gold.

Fabric

smart_device_analythcs_ws

Área de trabajo para el curso de Fabric en Proyecto 1

Ver canalización de implementación

Crear aplicación

Administrar acceso

Configuración del área de trabajo

+ Nuevo elemento

Nueva carpeta

Importar

Migrar

Filtrar por palabra cl...

Filtro

Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
Copy Job		Carpeta	—	—	—	—	—	—	
DataFlow_Gen2	...	Carpeta	—	—	—	—	—	—	
Notebook		Carpeta	Carpeta	—	—	—	—	—	
Pipeline		Carpeta	—	—	—	—	—	—	
lh_bronze		Lakehouse	Bronze D...	JESUS LIBRA...	—	—	—	—	
lh_bronze		Punto de con...	Bronze D...	JESUS LIBRA...	—	—	—	—	
lh_gold		Lakehouse	Gold Data	JESUS LIBRA...	—	—	—	—	
lh_gold		Punto de con...	Gold Data	JESUS LIBRA...	—	—	—	—	
lh_silver		Lakehouse	Silver Data	JESUS LIBRA...	—	—	—	—	
lh_silver		Punto de con...	Silver Data	JESUS LIBRA...	—	—	—	—	
sm_smarth_device_lh		Modelo semá...	—	smart_device...	17/8/2026, 11:2...	N/D	—	—	
StagingLakehouseForDataflows_202607310412:		Lakehouse	—	JESUS LIBRA...	—	—	—	—	

Creación de carpeta Pipeline/Canalizaciones para mantener la trazabilidad y el orden de flujo de trabajo.

Fabric

smart_device_analythcs_ws

Área de trabajo para el curso de Fabric en Proyecto 1

+ Nuevo elemento Nueva carpeta Importar Migrar

Ver canalización de implementación Crear aplicación Administrar acceso Configuración del área de trabajo

smart_device_analythcs_ws > Pipeline

Nombre	Estado	Tipo	Tarea	Propietario	Actualizado	Próxima actualización	Aprobación	Confidencial...	Incluido en la aplicación
Pi_ingest_smarth_device		Canalización	Process G...	JESUS LIBRA...	—	—	—	—	
Pi_process_smarth device		Canalización	Process G...	JESUS LIBRA...	—	—	—	—	
pi_transformation_smarth_device		Canalización	Process G...	JESUS LIBRA...	—	—	—	—	

Archivo de Pipeline utilizado para la automatización y canalización de los flujos de ingesta y transformación.

The screenshot displays the Microsoft Fabric interface for a pipeline named 'Pi_ingest_smarth_device'. The pipeline consists of two main activities: 'Get Folder Details' and 'If folder Exists'. The 'If folder Exists' activity has two branches: a 'True' branch with 9 activities (including 'Ingest Device File' and 'Agregar') and a 'False' branch with no activities. Below the pipeline, the 'Parámetros' (Parameters) tab is active, showing a table with two parameters: 'p_file_date' and 'p_environment', both of type 'Cadena' (String).

Nombre	Tipo	Valor predeterminado
p_file_date	Cadena	2025-07-07
p_environment	Cadena	Development

Demostración de parametrización dinámica a nivel de Pipeline basada en el campo `file_date`, facilitando la ejecución de cargas por lotes (*batch processing*) orientadas a fechas específicas, junto con el modelado de agregaciones por departamento para potenciar el análisis estructurado en las capas del Lakehouse.

The screenshot displays the Microsoft Fabric Data Factory interface. The top navigation bar includes the 'Fabric' logo, a search bar, and user information. The left sidebar shows various workspace icons. The main canvas features a pipeline with a 'Get Folder Details' activity connected to an 'If folder...' conditional logic block. The 'If folder...' block has two paths: 'True' (9 actividades) and 'False' (No hay ninguna actividad). The 'True' path contains an 'Ingest Device File' activity. The bottom pane shows the configuration for the 'If folder...' activity, with the expression '@activity('Get Folder Details').output...' and a table of activities for each path.

Escenario	Actividad
True	Ingest Device File
False	No hay ninguna actividad.

Creación de estructura condicional para minimizar errores con validacion “True” en caso de apuntar a Notebooks y no a carpetas vacías, albergando 9 Notebooks de ingesta (En caso de entrar en “False” se puede configurar una alerta a email.) Además se puede crear Schedule pero por temas demostrativos no se aplicó.

The screenshot displays the Microsoft Fabric user interface. At the top, the browser tab is labeled 'pi_transformation_smarth_device'. The main navigation bar includes 'Inicio', 'Actividades', 'Ejecutar', and 'Ver'. Below this, a secondary bar contains icons for 'Validar', 'Ejecutar', 'Programación', 'Desencadenador', 'Ver historial de ejecución', 'Copiar datos', 'Flujo de datos', and other tools. The central workspace shows a data pipeline with several 'DF Transformation' and 'Transformation' blocks connected sequentially. On the right side, there are zoom controls (plus, minus, and a vertical slider). The bottom panel is titled 'General' and 'Configuración'. It includes fields for 'Nombre' (set to 'DF Transformation Brand'), 'Descripción', 'Estado de la actividad' (set to 'Activado'), 'Tiempo de espera agotado' (set to '0.12:00:00'), and 'Habilitar reintentos'. A 'Más información' link is also present.

Implementación de pipeline en Microsoft Fabric combinando Dataflows y Notebooks para la transformación y limpieza iterativa de archivos hacia la capa Silver. Se configuraron parámetros dinámicos basados en file_date para el control de particiones. Ejecución documentada y validada bajo restricciones de capacidad de la licencia Trial."

Fabric **Pl_process_smarth device** X

Inicio Actividades Ejecutar Ver

Validar Ejecutar Programación Desencadenador Ver historial de ejecución Copiar datos Flujo de datos

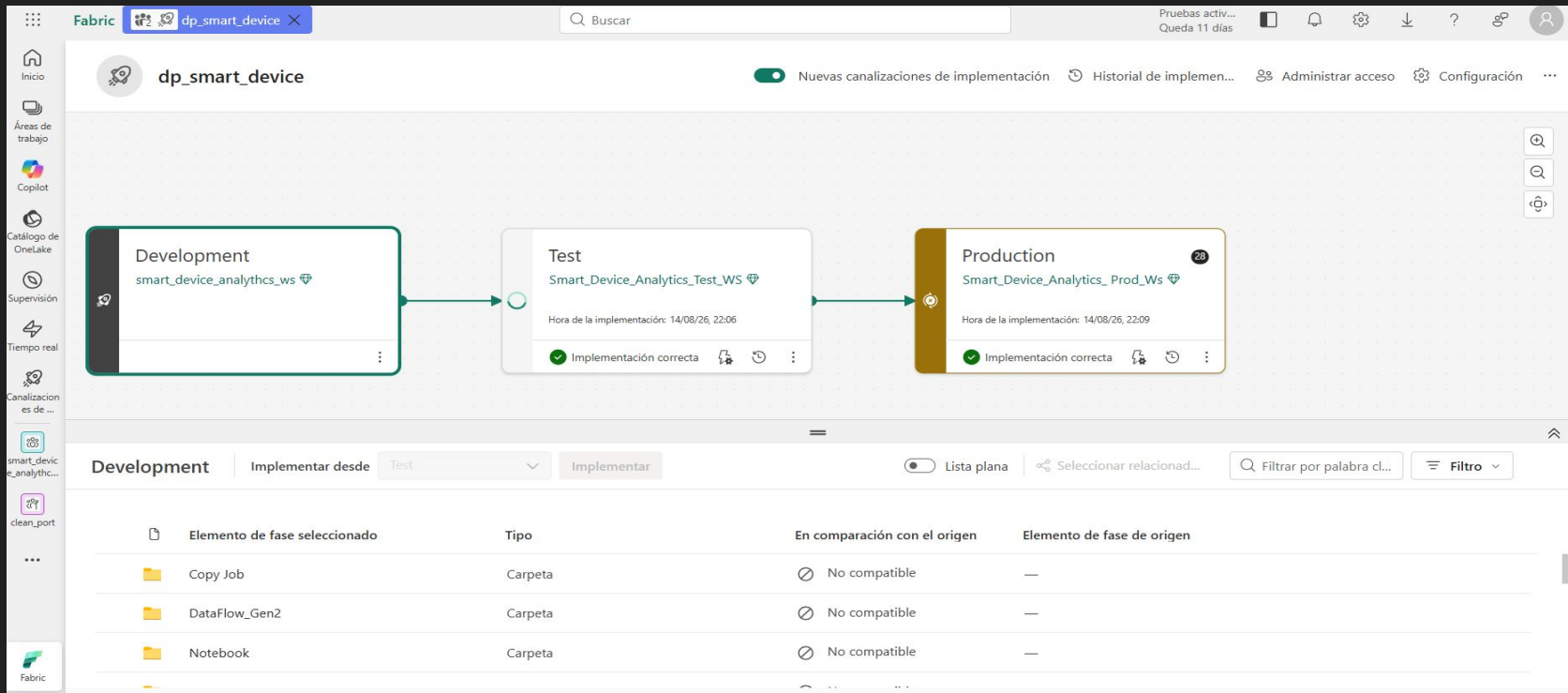
Invoke ingestion → Invoke transformation

Parámetros Variables Configuración Salida Variables de biblioteca

+ Nuevo Eliminar

☐	Nombre	Tipo	Valor predeterminado	
☐	p_file_date	Cadena	2025-07-14	
☐	p_environment	Cadena	Develotment	

Orquestación del pipeline principal mediante actividades Invoke para invocación y resguardo de pipeline ingesta y pipeline transformación unificados por conexión de ejecución al terminar ingesta prosigue pipeline transformación para conllevar una trazabilidad y secuencia procesal en ejecución orientado a Medallion.



Orquestación de Deployment Pipelines para llevar un control secuencial de procesos basados y orientados a arquitectura Medallón (Bronze, Silver y Gold) en sus respectivas funciones.

Demostración técnica de limpieza
extra mediante otros métodos según
requerimientos.

Fabric **dp_smart_device** **files_inventory** Guardado Buscar Pruebas activ... Queda 7 días

Inicio Editar Herramientas con IA Ejecutar Ver Comentarios Historial Desarrollo Compartir

Áreas de trabajo Copilot Catálogo de OneLake Supervisión Tiempo real Real_Time_Intelligence smart_device_analytics... clean_port

Limpieza de archivos, inicialmente con "files_invesntory"

```
1 ##Carga de librerías para schema personalizado
2 from pyspark.sql.types import StructField, StructType, IntegerType, FloatType, StringType
3
```

[3] ✓ - Command executed in 217 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:40:55 p.m. PySpark (Python)

```
1 ##Definicion de Schema propio para definir tipos de datos a manejar
2 files_inventory_schema = StructType(fields = [
3     StructField("item_id", IntegerType(), False ),
4     StructField("quantity", IntegerType(), True),
5     StructField("price", FloatType(), True),
6     StructField("category", StringType(), True)
7 ])
```

[4] ✓ - Command executed in 199 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:40:55 p.m. PySpark (Python)

Importación de librerías StructField, StructType y tipologías de datos que contiene Dataset, la creación de esquema explícito permite tener un control de los tipos de datos que se manejan en el Dataset.

The screenshot displays the Microsoft Fabric Data Explorer interface. The top navigation bar includes tabs for 'Inicio', 'Ejecutar', and 'Ver'. The main workspace shows a PySpark notebook with three code cells:

```
1 ##Carga de DataSet mediante SparkReader
2 files_inventory_df = spark.read\
3     .option("header", True)\
4     .schema(files_inventory_schema)\
5     .csv("abfss://clean_port@onelake.dfs.fabric.microsoft.com/lh_td.Lakehouse/Files/generated_files_inventory_large.csv")
```

[5] ✓ - Command executed in 2 s 17 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:40:57 p.m. PySpark (Python) ✓

```
1 files_inventory_df.printSchema()
```

[6] ✓

```
root
|-- item_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- price: float (nullable = true)
|-- category: string (nullable = true)
```

```
1 ##Verificacion de carga de DataFrame y verificacion de problemas base a muestreo de 100
2 ##pero podrian ser 1000 tomando en cuenta que el dataset contiene 10000
3 display(files_inventory_df.limit(100))
```

[7] ✓

The bottom of the interface shows a table view with 4 columns and 100 rows selected. The status bar at the bottom indicates 'No conectado' and 'Autoguardado: activado'.

Carga de Dataset CSV mediante “spark.read”, carga de “headers”, carga de esquema explícito, carga de archivo con ruta “ABFS”, revisión de tipos como medida de corroboración y verificación, además de carga de Dataset con “display” y limitación a 100 registros solo para visualizar la carga exitosa.

Fabric dp_smart_device files_inventory Guardado Buscar Pruebas activ... Queda 7 días

Inicio Editar Herramientas con IA Ejecutar Ver Comentarios Historial Desarrollo Compartir

Áreas de trabajo Copilot Catálogo de OneLake Supervisión Tiempo real Real_Time_Intelligence smart_device_analytics... clean_port

Explorador

```
1 ##Limpieza
2 ##Relleno de vacios mediante .fillna
3 files_inventory_df_no_null = files_inventory_df.fillna({"quantity" : 0,
4                                                         "price" : 0.0,
5                                                         "category" : "Unknown"
6                                                         })
7
```

[0] ✓ PySpark (Python)

```
1 ##Verificacion de relleno de Null estandarizado para tabajar con 0 y con desconocido
2 display(files_inventory_df_no_null.limit(100))
```

[9] ✓ PySpark (Python)

Table + Nuevo gráfico 4 columnas, 100 filas

Vista de tabla Descargar Filtrar por palabra

	123 item_id	123 quantity	12F price	ABC category
1	5000	10	0.0	C
2	5001	20	-50.0	Unknown
3	5002	50000	10.0	D
4	5003	10	12.0	D
5	5004	100	10.0	C

Inspeccionar

No conectado Autoguardado: activado 1 de 18 celdas seleccionadas

Limpieza de no nulos en columnas con valor detectado con el uso de “fill.na”, asunción de “0” en caso de datos numéricos y “Unknown” en caso de datos cadena. Verificación de aplicación correcta a través de “display”.

Fabric dp_smart_device files_inventory Guardado Buscar Pruebas activ... Queda 7 días

Inicio Editar Herramientas con IA Ejecutar Ver Comentarios Historial Desarrollo Compartir

Áreas de trabajo Copilot Catálogo de OneLake Supervisión Tiempo real Real-Time Intelligence smart_device_analytic... clean_port

Explorador

```
1 ##Cambio de nombres de headers con capitalize y toDF
2 nuevos_nombres = [n.capitalize() for n in files_inventory_df_no_null.columns]
3 ##Utilización de bucle mediante capitalize apuntando el for a las columnas
4 files_inventory_df_no_null = files_inventory_df_no_null.toDF(*nuevos_nombres)
5 ##Reasignación de valores ya Capitalizados y aplicación de toDF para transformale de nuevo a DataFrame
```

[10] ✓ - Command executed in 214 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:01 p.m. PySpark (Python)

```
1 display(files_inventory_df_no_null)
```

[11] ✓ - Command executed in 1 s 320 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:03 p.m. PySpark (Python)

Table + Nuevo gráfico 4 columnas, 1000 filas

Vista de tabla Descargar Filtar por palabra Inspeccionar

	Item_id	Quantity	Price	Category
1	5000	10	0.0	C
2	5001	20	-50.0	Unknown
3	5002	50000	10.0	D
4	5003	10	12.0	D
5	5004	100	10.0	C
6	5005	0	-50.0	a
7	5006	0	12.0	A

No conectado Autoguardado: activado 1 de 18 celdas seleccionadas

Cambio de nombres en columna mediante el resguardo de una variable temporal y uso Capitalize(), utilización de bucle “for” para recorrer los “headers” del DataFrame, creación de nueva variable para reasignación de la variable temporal creada anteriormente con una aplicación de “toDF” en transformación a DataFrame, Verificación de cambios mediante “display”.

Fabric | **dp_smart_device** | **files_inventory** | Guardado | Buscar | Pruebas activ... Queda 7 días | Comentarios | Historial | Desarrollo | Compartir

Inicio | Editar | Herramientas con IA | Ejecutar | Ver

Ejecutar todo | Conectar | PySpark (Python) | Entorno | Valor predeterminado ... | Data Wrangler | Copilot

Explorador

- Inicio
- Áreas de trabajo
- Copilot
- Catálogo de OneLake
- Supervisión
- Tiempo real
- Real_Time_Intelligence
- smart_device_analytics...
- clean_port

```
1  ##limpieza de espacios en blanco
2  ##Importar libreria correspondiente
3  import pyspark.sql.functions as F

[12] ✓ - Command executed in 230 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:03 p.m. PySpark (Python)

...

1  ##Eliminacion de espacios extra o demas de Category al ser nuestra columna String
2  files_inventory_df_semi_clean = files_inventory_df_no_null.withColumn("Category", F.trim(F.col("Category")))

[13] ✓ - Command executed in 233 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:03 p.m. PySpark (Python)

...

1  ##Limpieza de repetidos no podemos borrar data por lo tanto lo que nos interesa
2  ##saber es si tenemos inconsistencia en pk algun valor repetido
3  ##Validacion
4  total_reg = files_inventory_df_semi_clean.count()
5  total_unicos = files_inventory_df_semi_clean.select("Item_id").distinct().count()
6  print(f"Total de registros: {total_reg}")
7  print(f"Total de items unicos: {total_unicos}")

[14] ✓ - Command executed in 2 s 74 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:05 p.m. PySpark (Python)

Total de registros: 10000
Total de items unicos: 10000

1  ##Validacion de tipos de datos final despues de transformaciones
```

Fabric | No conectado | Autoguardado: activado | 1 de 18 celdas seleccionadas

Importación de functions resguardadas en variable F para su manipulación según requerimientos, Limpieza de espacios en blanco en el Dataset, creación temporal de variable asignacion de DataFrame con contenido no nulo, uso de “.withColumn” para seleccionar la columna correspondiente con espacios en blanco, utilización de variable “F” para uso “Trim” acompañado de “col” para apuntar a la columna. Verificación en conteo de datos como total de registros además de creación de nueva variable que almacene los datos únicos uso de “select” acompañado de “distinct” y nuevamente count para que detecte aquellas Primary Key repetidas las cuales deben de ser un valor único, al tener los valores únicos iguales a los registros nos indica no se tienen valores de error de PK.

The screenshot displays the Microsoft Fabric workspace interface. The top navigation bar includes tabs for 'Inicio', 'Editar', 'Herramientas con IA', 'Ejecutar', and 'Ver'. The left sidebar shows the 'Explorador' (Explorer) view with various components like 'Áreas de trabajo', 'Copilot', 'Catálogo de OneLake', 'Supervisión', 'Tiempo real', 'Real Time Intelligence', 'Smart device analytics', and 'Clean port'. The main workspace area shows a PySpark script being executed. The script consists of three commands: importing PySpark SQL functions, filtering for negative price anomalies, and displaying the final DataFrame. The execution results show the command being executed successfully. Below the code, a table view displays the resulting DataFrame with columns 'Item_id', 'Quantity', 'Price', and 'Category'. The table contains four rows of data.

```
1 import pyspark.sql.functions as Q
[16] ✓ - Command executed in 232 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:06 p.m.

1 ##Separacion de datos negativos en precio
2 df_anomalias_negativas = files_inventory_df_semi_clean.filter(Q.col("Price") < 0)
3 print(f"Alerta de precios en salgo negativo: {df_anomalias_negativas}")
4 files_inventory_df_final = files_inventory_df_semi_clean.filter(Q.col("Price") >= 0)
[17] ✓ - Command executed in 212 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:06 p.m.
... Alerta de precios en salgo negativo: DataFrame[Item_id: int, Quantity: int, Price: float, Category: string]

1 display(files_inventory_df_final.limit(1000))
[18] ✓ - Command executed in 1 s 311 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:07 p.m.
```

Item_id	Quantity	Price	Category
5000	10	0.0	C
5002	50000	10.0	D
5003	10	12.0	D
5004	100	10.0	C

Detección de último momento de valores negativos aplicación de “.filter” con uso de “col” para apuntar a la columna y condicionar los valores menores a 0 y almacenarlos en una variable para su tratamiento. y los correctos se resguardan en el DataFrame final para su conversión a formato Delta. Verificación de cambios

The screenshot displays the Fabric IDE interface. The top bar includes the 'Fabric' logo, a search bar, and user information. The left sidebar contains navigation icons for 'Inicio', 'Áreas de trabajo', 'Copilot', 'Catálogo de OneLake', 'Supervisión', 'Tiempo real', 'Real Time Intelligence', 'smart_device_analytics...', and 'clean_port'. The main workspace shows a Python script in the 'files_inventory' file. The script performs data type validation for 'Item_id', 'Quantity', 'Price', and 'Category' using dictionaries and conditional logic. The output pane at the bottom shows the command execution status and the resulting print statements.

```
1  ##Validacion de tipos de datos final despues de transformaciones
2  t_a_1 = dict(files_inventory_df_semi_clean.dtypes)["Item_id"]
3  if t_a_1 == "int":
4      print("Tipo correcto de dato")
5  else:
6      print("Emergencia cambio el tipo de dato despues de las transformaciones")
7
8  t_a_2 = dict(files_inventory_df_semi_clean.dtypes)["Quantity"]
9  if t_a_2 == "int":
10     print("Tipo correcto de dato")
11 else:
12     print("Emergencia cambio el tipo de dato despues de las transformaciones")
13
14 t_a_3 = dict(files_inventory_df_semi_clean.dtypes)["Price"]
15 if t_a_3 in ("float","double"):
16     print("Tipo correcto de dato")
17 else:
18     print("Emergencia cambio el tipo de dato despues de las transformaciones")
19
20 t_a_4 = dict(files_inventory_df_semi_clean.dtypes)["Category"]
21 if t_a_4 == "string":
22     print("Tipo correcto de dato")
23 else:
24     print("Emergencia cambio el tipo de dato despues de las transformaciones")
```

[15] ✓ - Command executed in 250 ms by JESUS LIBRADO CHAVEZ HUERTA on 22/8/2026, 7:41:05 p.m.

...
Tipo correcto de dato
Tipo correcto de dato
Tipo correcto de dato
Tipo correcto de dato

14 de 18 celdas seleccionadas

Verificación extra de tipos de datos después de las modificaciones para asegurar la correspondencia de tipos a través de estructuras condicionales con un “else” en detección de cambio de tipo.

Screenshot of the Microsoft Fabric interface showing a PySpark script and a data table.

Explorador

- Inicio
- Áreas de trabajo
- Copilot
- Catálogo de OneLake
- Supervisión
- Tiempo real
- Real Time Intelligence
- Smart device analytics
- Lean port

Inicio | Editar | Herramientas con IA | Ejecutar | Ver

Comentarios | Historial | Desarrollo | Compartir

PySpark (Python) | Entorno | Valor predeterminado ... | Data Wrangler | Copilot

11	5013	100	0.0	A
12	5014	-5	20.0	B
13	5015	0	10.0	A
14	5017	-5	20.0	C
15	5019	0	10.0	A
16	5020	20	99999.0	a
17	5022	20	12.0	A

```
1 ##Guardar en formato delta
2 files_inventory_df_final.write.format("delta").mode("overwrite").saveAsTable("dim_files_inventory")
3
```

PySpark (Python)

Cerrar

No conectado | Autoguardado: activado | 14 de 18 celdas seleccionadas

Resguardo en formato Delta, Overwrite y guardado como tabla, (aplicar un merge en caso de carga incremental y posiblemente uso de append)